

RELATÓRIO DE AUDITORIA TÉCNICA, FORENSE E DE CONFORMIDADE

Avaliação de Segurança da Informação, LGPD, Stack Tecnológica e Arquitetura no projeto GlicoGuard

Prof. Dr. Fabiano Menegidio
Universidade de Mogi das Cruzes - UMC

Junho de 2026

Resumo

Este relatório apresenta uma auditoria técnico-científica de ponta a ponta do projeto GlicoGuard, aplicação web educacional voltada ao gerenciamento de contas, autenticação, privacidade e registros básicos associados ao cuidado glicêmico. A avaliação foi conduzida sob a perspectiva de Segurança da Informação, governança de dados pessoais, conformidade com a Lei Geral de Proteção de Dados Pessoais (LGPD) e boas práticas de engenharia de software. A análise considera o alinhamento com controles inspirados na ABNT NBR ISO/IEC 27001:2022, com ênfase em confidencialidade, integridade, disponibilidade e autenticidade. Foram examinados a stack tecnológica Java/Spring Boot, os mecanismos de autenticação, o uso de PBKDF2 com salt, segundo fator por e-mail, tokens temporários de recuperação de senha, snapshots cifrados em AES/GCM, auditoria encadeada por hash, fluxo de consentimento e direitos do titular. Também foram identificados riscos técnicos relevantes: segredo SMTP presente em arquivo de configuração com valor padrão, chave mestra de criptografia com fallback demonstrativo, HTTPS desabilitado por padrão, ausência de evidência completa de CSRF e dependência de persistência local/in-memory para partes críticas. O parecer conclui que o protótipo possui base arquitetural viável para fins acadêmicos e demonstrativos, mas requer remediações mandatórias antes de homologação em produção, principalmente em gestão de segredos, hardening HTTP, proteção antifraude, persistência transacional, verificação de dependências e formalização de testes de segurança.

1 Dados de Identificação, Stack Tecnológica e Contexto Arquitetural

1.1 Identificação do projeto

O projeto auditado é o **GlicoGuard**, sistema web em Java para apoio ao gerenciamento de contas, autenticação, consentimento LGPD, privacidade e registros básicos relacionados ao cuidado glicêmico. O repositório contém um módulo principal recomendado para desenvolvimento e execução, denominado `glicoguardbeta-main/`, além de documentação, artefatos de apresentação, arquivos protegidos locais e snapshots cifrados em `data/protected/`.

1.2 Componentes críticos identificados

- `GlicoGuardApplication.java`: ponto de entrada da aplicação Spring Boot.
- `WebController.java`: rotas web, formulários, login, 2FA, privacidade, exportação, consentimento e administração.

- `AuthenticationService.java`: cadastro, autenticação primária, geração e validação de 2FA, recuperação e redefinição de senha.
- `CryptoService.java`: PBKDF2, salt, tokens, códigos curtos, digest SHA-256 e criptografia AES/GCM.
- `ProtectedStorageService.java`: gravação de snapshots cifrados de usuários, auditoria e exportações.
- `SecurityWebFilter.java`: cabeçalhos HTTP de segurança, política CSP e bloqueio opcional de conexões sem HTTPS.
- `PrivacyService.java`: consentimento, listagem de dados coletados, exportação e explicações de finalidade.
- `UserStore.java` e `InMemoryDatabase.java`: armazenamento em memória e persistência protegida por snapshot.

1.3 Mapeamento detalhado da stack

- **Linguagem principal:** Java 21.
- **Framework:** Spring Boot 3.4.4, com Spring Web MVC.
- **Interface:** Thymeleaf, HTML server-side e CSS estático.
- **Comunicação por e-mail:** Spring Mail com SMTP Gmail.
- **Persistência declarada:** PostgreSQL via driver JDBC e Spring Data JPA no `pom.xml`; entretanto, parte relevante do domínio usa armazenamento em memória e snapshots cifrados locais.
- **Serialização:** Jackson com suporte a `JavaTimeModule`.
- **Criptografia e credenciais:** PBKDF2WithHmacSHA256 para senhas, salt individual de 16 bytes, AES/GCM/NoPadding para snapshots, SHA-256 para digest de tokens e comparação em tempo constante quando aplicável.
- **Build e testes:** Maven, Spring Boot Maven Plugin, JUnit 5/Spring Boot Test.
- **Ambiente de execução:** aplicação monolítica web em processo único, porta padrão 8080, com opção de e-mail real ou simulado em diretório local `sent-emails/`.

1.4 Análise da arquitetura de software

A arquitetura predominante é um **monólito web em camadas**, com separação entre controller, service, model, repository, config e util. Há boa intenção arquitetural na existência de serviços especializados, mas o controlador web concentra diversas responsabilidades de orquestração de formulário e sessão. O fluxo de dados inicia em formulários HTML, passa pelo `WebController`, alcança serviços de autenticação, privacidade, medicação ou administração, e persiste em estruturas de domínio mantidas pelo `UserStore`. Os snapshots cifrados são gravados em arquivos locais, o que reduz exposição em repouso, mas cria ponto crítico de disponibilidade e limita escalabilidade horizontal.

O acoplamento é moderado: `AuthService` funciona como fachada para autenticação, administração, privacidade e medicamentos, enquanto serviços internos dependem de `UserStore`, `CryptoService` e `EmailService`. As fronteiras lógicas existem, porém ainda não representam boundaries transacionais robustos. Como ponto crítico, a presença simultânea de JPA/PostgreSQL declarado e persistência local/in-memory sugere arquitetura híbrida ainda em transição.

1.5 Tipo de dados processados

O sistema processa dados cadastrais comuns e dados pessoais sensíveis associados à saúde. Foram identificados: nome, e-mail, CPF, data de nascimento, perfil de acesso, papel do usuário, vínculo paciente/cuidador, eventos de autenticação, códigos temporários, solicitações de exclusão, consentimentos, registros de medicamentos e trilhas de auditoria. Os dados de saída incluem telas administrativas, exportações do titular e snapshots cifrados locais.

1.6 Enquadramento regulatório pela LGPD

O projeto trata dados pessoais comuns e **dados pessoais sensíveis**, pois o contexto glicêmico, medicamentos e cuidado de saúde se enquadram no art. 5º, II, da LGPD, que define dado sensível como informação sobre saúde ou vida sexual, entre outras categorias. O tratamento exige base legal adequada, finalidade específica, minimização, segurança, transparência, registro de consentimento quando aplicável e mecanismos efetivos para direitos do titular. O risco regulatório mais relevante é a reidentificação por associação entre CPF, e-mail, vínculo de cuidador e registros de saúde, bem como vazamento de snapshots, logs ou e-mails de 2FA.

2 Avaliação de AI-Assisted Development & Metodologia de Desenvolvimento

2.1 Classificação metodológica

O repositório apresenta sinais de desenvolvimento **híbrido**: há estrutura típica de engenharia tradicional em camadas, classes com responsabilidades nomeadas e configurações de segurança explícitas, mas também existem indícios de evolução incremental rápida, documentação extensa em linguagem explicativa e duplicidade de abordagens de persistência. Não há evidência conclusiva de geração automatizada por LLM, porém a forma de comentários e textos de README sugere auxílio documental ou desenvolvimento assistido.

2.2 Evidências técnicas de qualidade e manutenibilidade

1. **Violação parcial de DRY e concentração:** `WebController.java` possui muitas rotas e regras de fluxo, elevando o risco de regressão e dificultando testes unitários isolados.
2. **Redundância arquitetural:** coexistem `Spring Data JPA/PostgreSQL` no `pom.xml` e `application.properties`, enquanto o código de domínio mantém `UserStore/InMemoryDatabase` e snapshots cifrados locais. Essa dualidade precisa ser resolvida antes de produção.
3. **Tratamento de exceções:** há tratamento de exceções específicas em vários fluxos, o que é positivo; contudo, mensagens de falha dependem do fluxo de UI e não substituem telemetria estruturada, correlação por `request-id` e alertas operacionais.
4. **Segurança aplicada:** existem mecanismos reais de PBKDF2, salt, 2FA, expiração de tokens, bloqueio por tentativas e cabeçalhos HTTP. Entretanto, segredos de SMTP e chave demonstrativa permanecem como riscos críticos de configuração.
5. **Documentação:** o README e arquivos de documentação apresentam boa descrição funcional, mas ainda faltam evidências formais de testes de intrusão, análise SAST/DAST, SBOM e revisão de dependências.

2.3 Code Quality / AI Usage Score

74/100 – O projeto demonstra maturidade acadêmica superior a um protótipo simples, com controles de autenticação, privacidade e criptografia. A pontuação é reduzida pela presença de segredo em configuração, fallback inseguro de chave mestra, persistência híbrida, ausência de hardening completo de produção e necessidade de testes de segurança automatizados. O uso potencial de AI-assisted development é avaliado como moderado, sem evidência direta de código gerado automaticamente, mas com sinais de documentação fortemente guiada por checklist.

3 Diagnóstico Baseado na Tríade SIA/CIDA & Scores

3.1 Confidencialidade

76/100 – Há PBKDF2WithHmacSHA256 com salt individual, AES/GCM para snapshots, cookies HttpOnly/SameSite e 2FA por e-mail. O principal déficit é operacional: senha SMTP com valor padrão em `application.properties`, fallback de chave AES demonstrativa e HTTPS desabilitado por padrão. Em produção, segredos devem estar em cofre, variáveis obrigatórias e rotação controlada.

3.2 Integridade

72/100 – A auditoria encadeada por hash e o uso de AES/GCM contribuem para integridade de registros protegidos. Ainda faltam controles como assinatura de exportações, versionamento transacional consistente, proteção contra replay de formulários, validação centralizada de DTOs e política explícita contra alteração direta dos arquivos locais.

3.3 Disponibilidade

61/100 – O monólito é simples e executável, porém depende de armazenamento local/in-memory para partes críticas, gravações síncronas em arquivos, SMTP externo e ausência de limites explícitos de payload. Em ambiente real, falhas de disco, lock de arquivos, indisponibilidade SMTP e falta de escalabilidade horizontal podem afetar continuidade.

3.4 Autenticidade

78/100 – O sistema implementa autenticação primária, 2FA, expiração de sessão, expiração de tokens, papéis de usuário e trilha de auditoria. A maturidade aumentaria com CSRF explícito, política de senha formal, MFA com canal alternativo, assinatura de commits/releases, rastreabilidade de deployments e RBAC centralizado via Spring Security.

4 Modelagem de Ameaças e Resiliência a Ataques

4.1 Superfície de ataque

A superfície de ataque inclui formulários públicos de cadastro, login, 2FA, recuperação de senha, redefinição de senha, rotas autenticadas de medicação, privacidade, exportação e rotas administrativas. Também há exposição indireta por SMTP, arquivos locais, dependências Maven e templates Thymeleaf.

4.2 Ameaças principais

- **Credential stuffing e força bruta:** mitigado parcialmente por bloqueio temporário, mas deve ser complementado por rate limit por IP, e-mail e fingerprint.
- **CSRF:** aplicações Spring MVC com formulários POST devem evidenciar token CSRF ativo e testes. A auditoria não encontrou classe de configuração Spring Security dedicada no pacote examinado.
- **Path Traversal:** `ProtectedStorageService.storeEncryptedExport(String filename, Object payload)` recebe nome de arquivo e resolve no diretório protegido. Caso o nome seja derivado de entrada do usuário, deve haver normalização, whitelisting e checagem de boundary.
- **Exfiltração de segredos:** credenciais SMTP e senha de banco em configuração podem vaziar via repositório, logs ou pacote de entrega.
- **Supply chain:** dependências Spring Boot, PostgreSQL e plugins Maven precisam de versionamento fixado por BOM, análise OWASP Dependency-Check ou similar e SBOM.
- **DoS lógico:** falta de limites de tamanho de formulário, número de registros, volume de exportação e frequência de e-mails pode permitir exaustão de CPU, memória, disco ou SMTP.

5 Auditoria Completa Baseada em Checklist de Requisitos

Tabela 1: Checklist técnico de requisitos de segurança, LGPD, documentação e apresentação

Nº	Requisito auditado	Status	Justificativa técnica
1.1	Uso de hash criptográfico seguro para senhas	[CONF.]	<code>CryptoService</code> usa <code>PBKDF2WithHmacSHA256</code> . Embora <code>Argon2/bcrypt</code> sejam preferíveis em muitos cenários modernos, <code>PBKDF2</code> é aceitável quando parametrizado.
1.2	Parâmetros de custo do hash configurados e justificados	[PARC.]	Iterações e tamanho de chave estão em propriedades, com 120000 iterações. Falta benchmark formal e justificativa de custo por ambiente.
1.3	Uso de salt criptográfico único por usuário	[CONF.]	O serviço gera salt aleatório de 16 bytes com <code>SecureRandom</code> a cada criação/alteração de senha.
1.4	Armazenamento correto do hash + salt	[CONF.]	<code>UserAccount</code> mantém <code>passwordHash</code> e <code>passwordSalt</code> ; senha em texto puro não é persistida.
1.5	Autenticação de dois fatores implementada	[CONF.]	Há geração de código temporário de 6 caracteres e envio por e-mail ou outbox local.
1.6	Validação do 2FA após autenticação primária	[CONF.]	A sessão definitiva é criada apenas após validação do desafio 2FA.
1.7	Fluxo de autenticação documentado	[CONF.]	README descreve login, senha, <code>PBKDF2</code> , 2FA, expiração e criação de sessão.
1.8	Evidências funcionais de autenticação	[PARC.]	Há classes de teste no projeto, mas o pacote de auditoria não demonstra prints/logs funcionais completos para todos os fluxos.
1.9	Sessões com tempo de expiração	[CONF.]	<code>server.servlet.session.timeout=15m</code> .
1.10	Invalidação de sessão no logout	[CONF.]	Rota de logout invalida a sessão HTTP.

Continua na próxima página

Nº	Requisito auditado	Status	Justificativa técnica
1.11	Proteção contra força bruta	[PARC.]	Há bloqueio por tentativas por conta. Falta rate limit por IP/fingerprint e proteção distribuída.
1.12	Justificativas técnicas documentadas	[PARC.]	Há documentação geral, mas faltam decisões formais de arquitetura e threat model versionado.
2.1	Recuperação de senha implementada	[CONF.]	Fluxo <code>/recuperar-senha</code> e <code>/redefinir-senha</code> existe.
2.2	Token criptograficamente seguro	[CONF.]	Token usa 32 bytes aleatórios via <code>SecureRandom</code> e Base64 URL-safe.
2.3	Token com tempo de expiração	[CONF.]	Expiração configurada em 10 minutos.
2.4	Token invalidado após uso	[CONF.]	O fluxo limpa token e expiração após redefinição bem-sucedida.
2.5	Falha correta para token expirado	[CONF.]	Há verificação de expiração e falha controlada.
2.6	Registro de solicitação de recuperação em log	[CONF.]	Eventos de recuperação são adicionados à auditoria.
2.7	Registro de sucesso/falha do processo	[CONF.]	O serviço registra falhas e sucesso na redefinição.
3.1	Comunicação protegida por TLS/HTTPS	[PARC.]	Existe filtro para exigir HTTPS, mas <code>glicoguard.security.require-https=false</code> por padrão.
3.2	Bloqueio de conexões não seguras	[PARC.]	Bloqueio existe quando habilitado. Deve ser obrigatório em produção.
3.3	Evidência de tráfego cifrado	[N.E.]	Não foram fornecidos prints, certificados, proxy reverso ou captura demonstrando TLS ativo.
3.4	Dados sensíveis criptografados em repouso	[PARC.]	Snapshots em <code>data/protected</code> são cifrados; a cobertura em PostgreSQL/JPA não foi demonstrada.
3.5	Algoritmo criptográfico adequado	[CONF.]	AES/GCM/NoPadding com IV de 12 bytes e tag de 128 bits é adequado para confidencialidade e integridade autenticada.
3.6	Chaves criptográficas protegidas	[N.C.]	Há fallback <code>glicoguard-demo-master-key-2026-change-me</code> ; chaves devem ser obrigatórias fora do código e rotacionáveis.
3.7	Estratégia de criptografia documentada	[PARC.]	README e documentação citam PBKDF2/AES, mas faltam rotação, custódia, backup e resposta a comprometimento.
3.8	Justificativa técnica das escolhas	[PARC.]	Há descrição técnica, porém não há parecer de seleção de algoritmos e parâmetros.
4.1	Listagem completa dos dados pessoais coletados	[CONF.]	<code>PrivacyService</code> e README listam dados como nome, e-mail, CPF, nascimento, papel e registros.
4.2	Associação de cada dado a uma finalidade	[CONF.]	A área de privacidade associa dados a login, 2FA, cuidado, vínculo e segurança.
4.3	Evidência de minimização de dados	[PARC.]	Há preocupação documental; CPF e data de nascimento exigem justificativa forte e minimização granular.
4.4	Registro explícito de consentimento	[CONF.]	Modelo contém versão, finalidade e data de consentimento.
4.5	Consentimento associado à finalidade	[CONF.]	Propriedades registram versão e finalidade do consentimento.
4.6	Possibilidade de revogação do consentimento	[CONF.]	Há rota de revogação e registro associado.
4.7	Registro de data e versão do consentimento	[CONF.]	Campos <code>consentVersion</code> , <code>consentPurpose</code> e <code>datas</code> estão no domínio.

Continua na próxima página

Nº	Requisito auditado	Status	Justificativa técnica
4.8	Consulta aos dados do titular	[CONF.]	Área de privacidade exibe dados coletados.
4.9	Exportação dos dados	[CONF.]	Há rota /dados/exportar e geração de payload.
4.10	Exclusão dos dados pessoais	[PARC.]	Usuário solicita exclusão e administrador executa. Falta evidência de SLA e eliminação em backups/snapshots.
4.11	Fluxo de direitos documentado	[PARC.]	README descreve o fluxo, mas falta procedimento formal operacional.
5.1	Logs de autenticação registrados	[CONF.]	Eventos de login são registrados em auditoria.
5.2	Logs de falhas e 2FA registrados	[CONF.]	Falhas de senha, 2FA e recuperação são auditadas.
5.3	Proteção contra alteração dos logs	[PARC.]	Auditoria encadeada por hash e snapshot cifrado ajudam; falta WORM, assinatura externa ou trilha imutável.
5.4	Exemplo de análise de logs apresentado	[PARC.]	O sistema possui tela de auditoria, mas falta relatório analítico com cenários e evidências.
6.1	Documento de visão geral do sistema	[CONF.]	README e documentação fornecem visão geral.
6.2	Diagrama de arquitetura	[CONF.]	Há artefato de diagrama em docs/Diagrama de Arquitetura.jpg.
6.3	Fluxos de autenticação e dados documentados	[CONF.]	README descreve autenticação, 2FA, consentimento e privacidade.
6.4	Gestão de credenciais documentada	[PARC.]	Há propriedades de configuração, mas a política de segredos ainda é insuficiente.
6.5	Uso de criptografia documentado	[CONF.]	README e código evidenciam PBKDF2 e AES/GCM.
6.6	Identificação dos ativos do sistema	[PARC.]	Ativos principais estão descritos, mas falta inventário formal com criticidade e proprietário.
6.7	Identificação de ameaças e vulnerabilidades	[PARC.]	Há documentação de segurança, mas threat model formal STRIDE/LINDDUN não foi evidenciado.
6.8	Associação risco x contramedida	[PARC.]	Este relatório apresenta a associação; no repositório, a rastreabilidade ainda é parcial.
6.9	Testes de segurança realizados	[PARC.]	Existem testes funcionais de segurança, mas não há evidência completa de SAST/DAST/dependency scan.
6.10	Resultados dos testes documentados	[N.E.]	Não foi localizado relatório de execução de testes com evidências.
6.11	Uso de artigos científicos e/ou normas técnicas	[PARC.]	Documentação menciona LGPD e boas práticas; referências acadêmicas precisam ser normalizadas.
6.12	Referências normalizadas	[PARC.]	Faltam referências completas em padrão ABNT no material principal.
7.1	Resumo entre 200 e 300 palavras	[CONF.]	Há arquivo docs/resumo.md. Deve ser validado quanto à contagem final.
7.2	Objetivo claramente definido	[CONF.]	Objetivo do sistema e da segurança está descrito.
7.3	Metodologia técnica descrita	[CONF.]	Documentação aborda stack, fluxos e implementação.
7.4	Mecanismos de segurança apresentados	[CONF.]	PBKDF2, salt, 2FA, recuperação, consentimento e criptografia são apresentados.
7.5	Conformidade com a LGPD explicada	[CONF.]	README e telas de privacidade documentam princípios e direitos.
7.6	Terminologia técnica adequada	[CONF.]	A terminologia é adequada ao nível acadêmico.
7.7	Qualidade textual e científica	[PARC.]	Texto está claro, mas pode ganhar rigor com citações ABNT e metodologia de validação.

Continua na próxima página

Nº	Requisito auditado	Status	Justificativa técnica
8.1	Estrutura científica do pôster	[CONF.]	Há pôster em docs/Poster GlicoGuard.png.
8.2	Arquitetura e fluxos representados visualmente	[PARC.]	Há diagrama, mas a auditoria não valida integralmente a legibilidade do pôster.
8.3	Evidência de conformidade com LGPD	[PARC.]	LGPD é apresentada nos documentos; faltam evidências visuais detalhadas no material analisado.
8.4	Qualidade técnica dos diagramas	[PARC.]	Diagrama existe, mas precisa versionamento e aderência à arquitetura real final.
8.5	Coerência com o sistema entregue	[PARC.]	Conteúdo geral é coerente, mas persistência JPA versus in-memory precisa ser alinhada.
8.6	Domínio técnico na apresentação	[N.E.]	Critério depende de avaliação oral presencial; não é verificável apenas pelo repositório.
8.7	Capacidade de resposta às perguntas	[N.E.]	Critério depende de banca/apresentação; não é verificável por auditoria documental.

6 Matriz de Risco, Plano de Ação e Remediação

Tabela 2: Matriz de riscos técnicos priorizados

Risco/Vulnerabilidade	Pilar	Sev.	Impacto técnico	Plano de ação e remediação
Segredo SMTP e senha de banco com valores padrão em configuração	C/A	Crítica	Vazamento de credenciais, envio indevido a infraestrutura.	Remover segredos do repositório, exigir variáveis de ambiente, usar e-mails e cofre de segredos, rotação imediata e bloqueio de inicialização sem credenciais válidas.
Fallback de chave AES demonstrativa	C/I	Alta	Qualquer artefato por terceiros com acesso ao código.	Tornar GLICOGUARD_AES_KEY obrigatório, usar KMS/secret manager, padrão pode ser versionado e registrar eventos de troca de chave.
HTTPS desabilitado por padrão e CSRF não evidenciado	C/I/A	Alta	Sessões, formulários e tokens podem ser explorados em ambiente sem TLS/CSRF.	Habilitar HTTPS obrigatório em produção, Spring Security com CSRF para formulários, HSTS e cookies Secure.
Persistência local/in-memory e gravação síncrona de snapshots	D/I	Média	Perda de dados, inconsistência, dificuldade de sensíveis e dependência do sistema de arquivos local.	Consolidar persistência transacional no PostgreSQL, cifrar campos sensíveis, usar migrations e backups testados.

6.1 Exemplos práticos de remediação segura em Java/Spring

6.1.1 Tratamento seguro de caminhos e arquivos

Listing 1: Mitigação de Path Traversal com boundary check

```
1 @Service
2 public class SafeProtectedStorage {
3     private final Path baseDir = Path.of("data",
4         "protected").toAbsolutePath().normalize();
5
6     public Path resolveSafeFile(String requestedName) {
7         if (requestedName == null ||
8             !requestedName.matches("[a-zA-Z0-9._-]{1,80}\\\\.enc")) {
9             throw new IllegalArgumentException("Nome de arquivo invalido.");
10        }
11        Path target = baseDir.resolve(requestedName).normalize();
12        if (!target.startsWith(baseDir)) {
13            throw new SecurityException("Tentativa de path traversal bloqueada.");
14        }
15        return target;
16    }
17 }
```

6.1.2 Chave criptográfica obrigatória e sem fallback inseguro

Listing 2: Validação obrigatória de segredo criptográfico

```
1 private byte[] loadAesKeyFromEnvironment() {
2     String encodedKey = System.getenv("GLICOGUARD_AES_KEY_B64");
3     if (encodedKey == null || encodedKey.isBlank()) {
4         throw new IllegalStateException("GLICOGUARD_AES_KEY_B64 deve ser configurada em
5         cofre de segredos.");
6     }
7     byte[] key = Base64.getDecoder().decode(encodedKey);
8     if (key.length != 32) {
9         throw new IllegalStateException("A chave AES-256 deve possuir 32 bytes.");
10    }
11    return key;
12 }
```

6.1.3 Middleware para limitação de recursos

Listing 3: Filtro para limitar tamanho de requisição

```
1 @Component
2 public class RequestSizeLimitFilter extends OncePerRequestFilter {
3     private static final long MAX_BYTES = 256 * 1024;
4
5     @Override
6     protected void doFilterInternal(HttpServletRequest request,
7         HttpServletResponse response,
8         FilterChain chain) throws ServletException,
9         IOException {
10        long size = request.getContentLengthLong();
11        if (size > MAX_BYTES) {
12            response.sendError(HttpServletResponse.SC_REQUEST_ENTITY_TOO_LARGE,
13                "Requisicao excede o limite permitido.");
14        }
15    }
16 }
```

```
15     chain.doFilter(request, response);
16   }
17 }
```

6.1.4 Configuração segura de produção

Listing 4: Exemplo de application-prod.properties sem segredos fixos

```
1 glicoguard.security.require-https=true
2 server.servlet.session.cookie.secure=true
3 server.servlet.session.cookie.http-only=true
4 server.servlet.session.cookie.same-site=strict
5 spring.mail.username=${GLICOGUARD_MAIL_USERNAME}
6 spring.mail.password=${GLICOGUARD_MAIL_PASSWORD}
7 spring.datasource.url=${GLICOGUARD_DB_URL}
8 spring.datasource.username=${GLICOGUARD_DB_USER}
9 spring.datasource.password=${GLICOGUARD_DB_PASSWORD}
```

6.1.5 Gestão de dependências e verificação de supply chain

Listing 5: Exemplo de plugin Maven para auditoria de dependências

```
1 <plugin>
2   <groupId>org.owasp</groupId>
3   <artifactId>dependency-check-maven</artifactId>
4   <version>12.1.0</version>
5   <configuration>
6     <failBuildOnCVSS>7</failBuildOnCVSS>
7     <format>ALL</format>
8   </configuration>
9   <executions>
10    <execution>
11      <goals>
12        <goal>check</goal>
13      </goals>
14    </execution>
15  </executions>
16 </plugin>
```

7 Conclusão do Parecer de Auditoria & Referências Bibliográficas

7.1 Parecer conclusivo

O GlicoGuard é tecnicamente viável como protótipo acadêmico avançado e demonstra compreensão relevante de controles de segurança: autenticação com 2FA, hashing com salt, expiração de sessão, recuperação de senha temporária, consentimento, direitos do titular, trilha de auditoria e cifragem de snapshots. Entretanto, o software ainda não deve ser homologado como aplicação produtiva de saúde sem correções mandatórias. A prioridade deve recair sobre: remoção de segredos do repositório, exigência de TLS, gestão de chave criptográfica sem fallback, validação CSRF, rate limit robusto, consolidação da persistência, revisão da exposição de dados sensíveis, implantação de SAST/DAST/dependency scan e formalização de evidências de testes.

A arquitetura em camadas é adequada para o porte do projeto, desde que a fronteira entre persistência local e banco relacional seja resolvida. Para produção, recomenda-se Spring Security completo, PostgreSQL transacional com migrações, criptografia de campos sensíveis, cofre de segredos, observabilidade estruturada, logs imutáveis, backup cifrado, política de retenção e documentação formal de DPIA/RIPD conforme risco do tratamento de dados de saúde.

7.2 Referências bibliográficas

- ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **ABNT NBR ISO/IEC 27001:2022**: Segurança da informação, segurança cibernética e proteção à privacidade - Sistemas de gestão da segurança da informação - Requisitos. Rio de Janeiro: ABNT, 2022.
- BRASIL. **Lei nº 13.709, de 14 de agosto de 2018**. Lei Geral de Proteção de Dados Pessoais (LGPD). Diário Oficial da União, Brasília, DF, 2018.
- OWASP FOUNDATION. **OWASP Application Security Verification Standard (ASVS)**. OWASP, versão 4.x.
- OWASP FOUNDATION. **OWASP Top 10 Web Application Security Risks**. OWASP, edição vigente.
- SPRING. **Spring Boot Reference Documentation**. VMware Tanzu/Spring, documentação técnica oficial.
- NIST. **Digital Identity Guidelines**. NIST Special Publication 800-63, edição vigente.